AI110 | Foundations of AI Engineering

Foundations of AI Engineering Spring 2026 (@ Section 1C | Wednesdays 6PM - 8PM PT)
Personal Member ID#: **125461**

Welcome

Getting Started

▶ Course Info

AI Prompting Guide

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

📢 Reminder: Project 2 is due by **Monday, March 30th at 1:59AM CDT**.

Submit

Show What You Know: PawPal+

ℹ️ Project Overview

⏰ ~4 hours

You've been asked to design PawPal+, a smart pet care management system that helps owners keep their furry friends happy and healthy. The app will track daily routines -- feedings, walks, medications, and appointments -- while using algorithmic logic to organize and prioritize tasks.

Your mission is to move from concept to a working application by designing a modular system architecture using Python's object-oriented programming (OOP). You will act as the lead architect, using AI to brainstorm your design, scaffold your core logic, and implement sophisticated scheduling algorithms. You will practice a "CLI-first" workflow, ensuring your backend logic in `pawpal_system.py` is robust and verified through a demo script before connecting it to a modern Streamlit UI.

🎯 Goals

By completing this project, you will be able to...

- [ ] Design a modular system using Python classes and visualize relationships with AI-generated Mermaid.js UML diagrams.
- [ ] Implement OOP principles to represent real-world entities like `Owner`s, `Pet`s, and `Task`s.
- [ ] Develop algorithmic logic for sorting, conflict detection, and recurring task management.
- [ ] Verify system behavior through a CLI demo script and automated `pytest` suites generated with AI assistance.
- [ ] Communicate design intent and reflect on AI-human collaboration tradeoffs in a professional README and consolidated reflection.

✏️ Project Instructions

▾ Phase 1: System Design with UML + AI Support

⏰ **~45 mins**

In this phase, you'll design the structure of your PawPal+ app before writing any code. You'll use AI to help brainstorm and visualize the main components (Owner, Pet, Task, Scheduler) and how they connect in a simple UML diagram.

Step 1: Understand the Problem

- [ ] Go to the **PawPal+** repo.
- [ ] Click **Fork** to create your own copy under your GitHub account, then clone your fork to your computer.
- [ ] Clone the fork to your computer, then open the cloned folder in VS Code.
- [ ] Read the scenario in `README.md` to understand what PawPal+ is intended to do.

- [ ] Identify **three core actions** a user should be able to perform (e.g., add a pet, schedule a walk, see today's tasks).
- [ ] Open `reflection.md` and document these actions in natural language under the **System Design** section.

Step 2: List the Building Blocks

- [ ] Brainstorm the main objects needed for the system. For each object, determine:
  - What information it needs to hold (**attributes**)
  - What actions it can perform (**methods**)

Step 3: Draft Your UML with AI

- [ ] Open the Copilot Chat in VS Code.
- [ ] Mention you are designing a pet care app with the four classes identified above.
- [ ] Ask Copilot to create a Mermaid.js class diagram based on your brainstormed attributes and methods.

  > Mermaid is a text-based tool that renders diagrams. You can preview Mermaid code in VS Code or paste it into the [Mermaid Live Editor]() to see your chart.

- [ ] Review the diagram. Ensure relationships (like "Owner has Pets") make sense and that you haven't included unnecessary complexity.
- [ ] *Keep this diagram handy - we'll revisit it in Phase 6!*

Step 4: Translate UML into a Skeleton

- [ ] Create a new file named `pawpal_system.py`. This will be your "logic layer" where all your backend classes live.

- [ ] Use **Inline Chat** or **Agent mode** to generate the "skeleton" of your classes (the names, attributes, and empty method stubs) based on your UML.

  Tell Copilot to use **Python Dataclasses** for objects like `Task` and `Pet` to keep your code clean.

- [ ] Commit your draft to GitHub: `git commit -m "chore: add class skeletons from UML"`.

Step 5: Reflect and Refine

- [ ] Open `reflection.md`.
- [ ] Answer section "1a. Initial design" by describing the classes you chose and their responsibilities.
- [ ] Ask Copilot to review your skeleton: Use `#file:pawpal_system.py` and ask if it notices any missing relationships or potential logic bottlenecks.
- [ ] If you make changes based on AI feedback, document what you changed and why in section "1b. Design changes".

📍 **Checkpoint**: You've created a clear UML diagram and matching Python class skeletons in `pawpal_system.py`! Your system's blueprint is complete and ready for implementation.

Close Section

▾ Phase 2: Core Implementation

⏰ **~90 mins**

In this phase, you'll translate your UML design into working Python code. You'll follow a "CLI-first" workflow, meaning you'll build and verify your backend logic in a standalone script before touching the

Streamlit UI. This ensures your system's "brain" is solid.

Step 1: Scaffold the Logic Layer

Now you'll write the full code for all your main classes.

- [ ] Use **Agent Mode** to flesh out the core implementation of your four classes in `pawpal_system.py`
  - [ ] `Task`: Represents a single activity (description, time, frequency, completion status).
  - [ ] `Pet`: Stores pet details and a list of tasks.
  - [ ] `Owner`: Manages multiple pets and provides access to all their tasks.
  - [ ] `Scheduler`: The "Brain" that retrieves, organizes, and manages tasks across pets.

    If you aren't sure how a `Scheduler` should "talk" to an `Owner` to get pet data, ask Copilot: "Based on my skeletons in `#file:pawpal_system.py`, how should the `Scheduler` retrieve all tasks from the `Owner`'s pets?"

Step 2: Create and Run a Demo Script

- [ ] Create a new file named `main.py`. This is your temporary "testing ground" to verify your logic works in the terminal.
- [ ] Write a script in `main.py` that performs the following:
  - [ ] Imports your classes from `pawpal_system.py`.
  - [ ] Creates an `Owner` and at least two `Pet`s .
  - [ ] Adds at least three `Task`s with different times to those pets.
  - [ ] Prints a "Today's Schedule" to the terminal.

- [ ] Run your script: `python main.py`.

  If your schedule prints out as a messy list of objects, use **Inline Chat** on your print statement and ask: "Suggest a clearer, more readable way to format this schedule output for the terminal".

Step 3: Add Quick Tests

- [ ] Open the terminal and ensure you have `pytest` installed (`pip install pytest`).
- [ ] Create a file named `tests/test_pawpal.py`.
- [ ] Use the **Generate tests** smart action or Copilot Chat to draft two simple tests:
  - [ ] **Task Completion:** Verify that calling `mark_complete()` actually changes the task's status.
  - [ ] **Task Addition:** Verify that adding a task to a `Pet` increases that pet's task count.
- [ ] Run your tests by typing `python -m pytest` in the terminal.

Step 4: Document, Reflect, and Merge

- [ ] Use the **Generate documentation** smart action to add 1-line docstrings to your methods in `pawpal_system.py`
- [ ] Use Copilot's **Generate Commit Message** feature in the Source Control tab to summarize your implementation.
- [ ] Push your work: `git push origin main`.

📍 **Checkpoint**: You've transformed your UML design into a functioning system! Your classes now work together to manage pets, tasks, and schedules, and you've verified them using a CLI demo script and initial automated tests.

Close Section

▾ Phase 3: UI and Backend Integration

⏰ **~20 mins**

Currently, your logic (`pawpal_system.py`) and your user interface (`app.py`) live in separate worlds. In this phase, you will act as the "bridge" to ensure that when a user clicks a button in the app, your Python classes actually respond.

Step 1: Establish the Connection

To use the `Owner`, `Pet`, and `Task` classes inside your Streamlit script, you must first make them accessible.

- [ ] Use a Python `import` statement to bring specific classes from `pawpal_system.py` into `app.py`.

Step 2: Manage the Application "Memory"

Streamlit is stateless, meaning it runs your code from top to bottom every time you click a button. If you simply create an `Owner` at the top of the script, it will be "reborn" (and empty) every time the page refreshes.

- [ ] Use AI to investigate `st.session_state`. Find out how to check if an object (like your `Owner` instance) already exists in the "vault" of the session before creating a new one.

  Think of `st.session_state` as a dictionary. You want to store your `Owner` object there so your data persists while you navigate the app.

Step 3: Wiring UI Actions to Logic

- [ ] Locate the UI components for "Adding a Pet" or "Scheduling a Task" in `app.py`. Replace those placeholders with calls to the methods you wrote in Phase 2.

  If a user submits a form to add a new pet, which class method should handle that data, and how does the UI get updated to show the change?

📍 **Checkpoint**: Your `app.py` successfully imports your logic layer! Adding a pet in the browser actually creates a `Pet` object that stays in memory.

Close Section

▾ Phase 4: Algorithmic Layer

⏰ **~45 mins**

Make your PawPal+ system *smart*! In this phase, you'll add simple algorithms that make your app more functional and intelligent -- sorting, filtering, recurring tasks, and basic conflict detection. You'll ask AI to brainstorm, write, and compare solutions, learning how to evaluate algorithmic choices for clarity and efficiency.

Step 1: Review and Plan

- [ ] Review your `main.py` demo from Phase 2. Identify where the current logic feels manual or overly simple.
- [ ] Open a **New Chat Session** to keep your algorithmic planning separate from your core implementation.
- [ ] Use `#codebase` to ask Copilot to suggest a list of small algorithms or logic improvements that could make your scheduling app more efficient for a pet owner.

- [ ] **Target Features:** You will implement logic for sorting tasks by time, filtering by pet/status, handling recurring tasks, and basic conflict detection .

Step 2: Implement Sorting and Filtering

- [ ] Open `pawpal_system.py`.
- [ ] **Sorting Logic:** Use **Inline Chat** on your `Scheduler.sort_by_time()` method to ask for a way to sort your `Task` objects by their time attribute.

  > Python's `sorted()` function is powerful. Ask Copilot how to use a `lambda` function as a "key" to sort strings in "HH:MM" format.

- [ ] **Filtering Logic:** Use **Agent Mode** to implement a method that filters tasks by completion status or pet name.
- [ ] Update your `main.py` to add tasks out of order, then print the results using your new sorting and filtering methods to ensure they work in the terminal.

Step 3: Automate Recurring Tasks

- [ ] Add logic to your `Task` or `Scheduler` class so that when a "daily" or "weekly" task is marked complete, a new instance is automatically created for the next occurrence.
- [ ] Use **Agent Mode** to handle this change, as it may require edits to how `mark_task_complete` interact with the `Task` frequency.

  > If a task happens "Daily," its new due date should be `today + 1 day`. Ask Copilot how to use Python's `timedelta` to calculate this accurately.

Step 4: Detect Task Conflicts

- [ ] Extend your `Scheduler` to detect if two tasks for the same pet (or different pets) are scheduled at the same time.
- [ ] Ask Copilot for a "lightweight" conflict detection strategy that returns a warning message rather than crashing the program.
- [ ] Update `main.py` with two tasks at the same time and verify that your `Scheduler` correctly identifies and prints a warning.

Step 5: Evaluate and Refine

- [ ] Share one of your completed algorithmic methods with Copilot and ask: "How could this algorithm be simplified for better readability or performance?".
- [ ] Review the AI's suggestion. If its version is more "Pythonic" but harder for a human to read, decide which version to keep.
- [ ] Open `reflection.md` and document one tradeoff your scheduler makes (e.g., only checking for exact time matches instead of overlapping durations) in section "2b. Tradeoffs".

Step 6: Document and Merge

- [ ] Use the **Generate documentation** smart action to add docstrings to your new algorithmic methods.
- [ ] Update your `README.md` with a short section called **Smarter Scheduling** summarizing your new features.
- [ ] Commit and push your changes directly to the `main` branch:

```
git add .
git commit -m "feat: implement sorting, filtering, and conflict detection"
git push origin main
```

📍 **Checkpoint**: You've added algorithmic intelligence to PawPal+! Your system can now sort, filter, detect conflicts, and handle recurring tasks, all verified through your CLI demo script.

Close Section

▾ Phase 5: Testing and Verification

⏰ **~30 mins**

In this phase, you'll test and verify that your PawPal+ system works as intended. You'll write and run simple tests to confirm that your classes, algorithms, and scheduling logic behave correctly, and use AI to help generate, explain, and review those tests.

Step 1: Plan What to Test

- [ ] Review your `pawpal_system.py` and list 3–5 core behaviors to verify.
- [ ] Start a New Chat Session in Copilot Chat to focus entirely on testing.
- [ ] Use `#codebase` to ask Copilot for a test plan: "What are the most important edge cases to test for a pet scheduler with sorting and recurring tasks?" .

  Focus on "happy paths" (everything works) and "edge cases" (e.g., a pet with no tasks, or two tasks at the exact same time).

Step 2: Build the Automated Test Suite

- [ ] Navigate back to `tests/test_pawpal.py`.
- [ ] Use the **Generate tests** smart action or Copilot Chat to draft your test functions.

- [ ] Ensure your suite includes at least:
  - [ ] **Sorting Correctness:** Verify tasks are returned in chronological order.
  - [ ] **Recurrence Logic:** Confirm that marking a daily task complete creates a new task for the following day.
  - [ ] **Conflict Detection:** Verify that the `Scheduler` flags duplicate times.

> Use **Ask mode** in Chat to explain any test code you don't understand before you save it.

Step 3: Run and Debug

- [ ] In your terminal, run your tests using: `python -m pytest`.
- [ ] If a test fails, use **Inline Chat** on the failing test and ask: "Why is this test failing, and is the bug in my test code or my `pawpal_system.py` logic?".
- [ ] Rerun `python -m pytest` until all tests pass with green checkmarks.

Step 4: Finalize Documentation and Merge

- [ ] Open your `README.md` and add a section titled "Testing PawPal+".
- [ ] Include the command to run tests (`python -m pytest`) and a brief description of what your tests cover.
- [ ] Provide your "Confidence Level" (1–5 stars) in the system's reliability based on your test results.
- [ ] Commit and push your test suite to the `main` branch:

```
git add .
git commit -m "test: add automated test suite for PawPal+ system"
git push origin main
```

📍 **Checkpoint**: You've built a robust test suite that verifies your system's intelligence! You've practiced using AI to generate and debug tests while maintaining the human oversight needed to ensure they are meaningful.

Close Section

▾ Phase 6: UI Polish, Documentation, and Reflection

⏰ **~30 mins**

In this final phase, you will package your PawPal+ project for others to understand and use. You'll ensure your UI accurately reflects the smart logic you built, finalize your system diagram, and complete a deep reflection on your AI-assisted engineering process.

Step 1: Reflect the Algorithmic Layer in the UI

Your backend is now "smart," but your UI might still be basic. Ensure the user can actually see and use the features you built in Phase 3.

- [ ] Update your display logic in `app.py` to use the methods from your `Scheduler` class (like sorting or conflict warnings).
- [ ] Use Streamlit components like `st.success`, `st.warning`, or `st.table` to make the sorted and filtered data look professional.

> If your `Scheduler` flags a task conflict, how should that warning be presented in the Streamlit UI to be most helpful to a pet owner?

Step 2: Finalize Your System Architecture (UML)

- [ ] Revisit the Mermaid.js UML diagram you drafted in Phase 1. Does it still match your final code in `pawpal_system.py`?
- [ ] Use Copilot with `#file:pawpal_system.py` and ask: "Based on my final implementation, what updates should I make to my initial UML diagram to accurately show how my classes interact?".
- [ ] Adjust your Mermaid code or draw.io diagram to reflect any new methods or relationships you added during the build.
- [ ] Save your final diagram as `uml_final.png` (or a similar image format) in your project folder.

Step 3: Polish Your README

- [ ] Open `README.md`. Your README should act as a professional manual for your app.
- [ ] Use Copilot with `#codebase` to help draft a "Features" list that accurately describes the algorithms you implemented (e.g., "Sorting by time," "Conflict warnings," "Daily recurrence").
- [ ] Include a screenshot of your final Streamlit app in the "📸 Demo" section.

  To embed your screenshot, use the Markdown syntax:
  `<a href="/course_images/ai110/your_screenshot_name.png" target="_blank"><img src='/cours`
  .

Step 4: Write Your Reflection

- [ ] Open `reflection.md`. You will complete the structured prompts covering your design choices, tradeoffs, and AI strategy.
- [ ] Reflect on AI Strategy: Specifically describe your experience with VS Code Copilot:
  - Which Copilot features were most effective for building your scheduler?
  - Give one example of an AI suggestion you rejected or modified to keep your system design clean.
  - How did using separate chat sessions for different phases help you stay organized?

- [ ] Summarize what you learned about being the "lead architect" when collaborating with powerful AI tools.

📍 **Checkpoint**: You've documented, reflected on, and finalized your PawPal+ project, transforming it from a coding exercise into a polished, professional artifact! You can now clearly explain your design, your reasoning, and your role as the human collaborator in an AI-assisted workflow.

Close Section

▾ Optional Extensions

⏰ **~30 mins**

Finished early or want to challenge yourself? Try extending PawPal+ with new features or smarter functionality. Each idea below builds on what you've already learned while letting you explore new tools or techniques.

- [ ] **Challenge 1: Advanced Algorithmic Capability via Agent Mode**
  - Add a third algorithmic capability (like "next available slot", weighted prioritization, etc.) that goes beyond the basic requirements.
  - In your `README.md`, document how **Agent Mode** was used to implement the logic.

- [ ] **Challenge 2: Data Persistence with Agent Mode**
  - Make your PawPal+ system remember pets and tasks between runs by saving them to a `data.json` file.
  - Use **Agent Mode** to plan and execute the changes. Tell the Agent: "Add `save_to_json` and `load_from_json` methods to the `Owner` class in `#file:pawpal_system.py`, then update the Streamlit state in `#file:app.py to load this data on startup".

    Persistence can be tricky with complex objects. Ask Copilot how to use a library like `marshmallow` or a custom dictionary conversion to handle JSON serialization.

- [ ] **Challenge 3: Advanced Priority Scheduling and UI**
  - Go beyond simple time sorting by adding **Priority-Based Scheduling**.
  - Add a "Priority" level (Low, Medium, High) to your `Task` class and update your Scheduler to sort by priority first, then by time.
  - Use the **Fix with Copilot** or **Inline Chat** features to add emojis or color-coding to your Streamlit table based on task priority (e.g., 🔴 for High, 🟡 for Medium).
- [ ] **Challenge 4: Professional UI and Output Formatting**
  - Add some user-friendly output such as color-coded status indicators, emojis for different task types, or structured CLI tables using libraries like `tabulate`.
  - This formatting should significantly improve the readability and the overall "feel" of the assistant.
- [ ] **Challenge 5: Multi-Model Prompt Comparison**
  - Choose a complex algorithmic task (like the logic for rescheduling weekly tasks) and ask for a solution from two different models (e.g., OpenAI vs. Claude or Gemini).
  - In your `reflection.md`, add a **Prompt Comparison** section documenting which model provided a more modular or "Pythonic" solution and why.

📍 **Checkpoint**: You've gone beyond the base requirements and made PawPal+ more capable and professional! These extensions demonstrate your ability to use AI for high-level system planning, data management, and user experience design.

Close Section

📬 Submitting Your Project

Once you've completed all the required features for your project, use the following checklist to prepare your work for submission.

1. Code is pushed to the correct GitHub repository
2. Repo is public
3. Required files are present (code, README, reflection, tests if applicable)
4. Commit history shows multiple meaningful commits
5. Reflection answers the prompts with specific, honest details
6. Final changes are committed and pushed before the deadline
7. Submit your assignment using the submit button.